

SEMANA 4

Trabajo práctico 04 - Aplicación web con EJS

1. Información y entrega

- **Modalidad:** individual.
- **Dominio obligatorio:** mascotas en adopción.
- **Entrega:** URL de un repositorio de GitHub.
- **Vencimiento:** lunes siguiente a las 19:59.
- **Ejecución esperada:** `npm install` y luego `npm start` .

El trabajo debe producir páginas HTML renderizadas. No alcanza con devolver JSON para los endpoints obligatorios.

2. Consigna

Crear una aplicación web para consultar mascotas en adopción y agregar temporalmente nuevos registros mediante un formulario.

La aplicación debe integrar:

- datos iniciales desde JSON;
- Express y EJS;
- layout principal;
- parciales de encabezado y pie;
- página inicial;
- listado y estado vacío;
- detalle y página HTML 404 ;
- formulario con validación mínima;
- recursos estáticos;
- creación en memoria y redirección.

3. Estructura obligatoria

```
tp-04-mascotas-ejs/  
|-- datos/  
|   |-- mascotas.json  
|-- public/  
|   |-- css/  
|   |   |-- estilos.css  
|   |-- img/  
|   |   |-- mascota.svg  
|   |-- js/  
|   |   |-- app.js  
|-- src/  
|   |-- archivos.js  
|   |-- index.js  
|-- views/  
|   |-- layouts/  
|   |   |-- main.ejs  
|   |-- partials/  
|   |   |-- encabezado.ejs  
|   |   |-- pie.ejs  
|   |-- mascotas/  
|   |   |-- lista.ejs  
|   |   |-- detalle.ejs  
|   |   |-- nueva.ejs  
|   |-- inicio.ejs  
|   |-- no-encontrado.ejs  
|-- .gitignore  
|-- package.json  
|-- package-lock.json  
|-- README.md
```

Se pueden agregar recursos o parciales, pero deben conservarse responsabilidades reconocibles.

4. Configuración

```
npm init -y  
npm install express ejs express-ejs-layouts
```

Configurar CommonJS y scripts:

```
"type": "commonjs",  
"scripts": {  
  "start": "node src/index.js",  
  "check": "node --check src/index.js && node --check src/archivos.js"  
}
```

Las tres dependencias deben aparecer en `dependencies` . `node_modules` debe quedar excluido.

5. Datos iniciales

Crear al menos cinco registros propios en `datos/mascotas.json` .

Propiedad	Tipo	Requisito
<code>id</code>	Número	Único y ordenado
<code>nombre</code>	Texto	Nombre de la mascota
<code>especie</code>	Texto	Por ejemplo perro o gato
<code>edad</code>	Número	Cero o mayor

Propiedad	Tipo	Requisito
descripcion	Texto	Presentación breve
estado	Texto	En adopción , Reservada o Adoptada
imagen	Texto	Ruta pública /img/mascota.svg

El SVG puede ser una ilustración sencilla propia o un recurso proporcionado para la actividad. No utilizar imágenes sin permiso ni enlaces externos cuya disponibilidad no se controle.

6. Configuración de Express y EJS

Después de cargar el JSON, la aplicación debe configurar:

```
app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "..", "views"));

app.use(expressLayouts);
app.set("layout", "layouts/main");
app.use(express.static(path.join(__dirname, "..", "public")));
app.use(express.urlencoded({ extended: false }));
```

El servidor debe comenzar a escuchar solamente después de que los datos iniciales se hayan leído correctamente.

7. Contrato obligatorio

```
GET /
GET /mascotas
GET /mascotas/nueva
GET /mascotas/:id
POST /mascotas
```

`GET /mascotas/nueva` debe declararse antes de `GET /mascotas/:id` .

Página inicial

Debe incluir:

- título principal;
- explicación breve del sitio;
- enlace visible al catálogo;
- estructura compartida del layout.

Listado

Debe:

- recibir el arreglo desde `res.render` ;
- recorrerlo mediante `forEach` ;
- mostrar una tarjeta por mascota;
- incluir nombre, especie, edad y estado;
- enlazar al detalle;
- mostrar un mensaje alternativo cuando la colección esté vacía;
- incluir enlace al formulario.

Detalle

Debe:

- buscar por identificador;
- mostrar todos los datos de la mascota;
- mostrar la imagen con texto alternativo apropiado;
- incluir enlace de regreso;
- responder 404 y renderizar `no-encontrado.ejs` cuando el ID no exista.

Formulario

Debe incluir controles etiquetados para:

- nombre;
- especie;
- edad;
- estado;
- descripción.

El formulario debe utilizar:

```
<form action="/mascotas" method="post">
```

Cada control debe tener `id` y `name` . Para `estado` se recomienda un `select` con los tres valores permitidos.

Procesamiento

En `POST /mascotas` :

1. leer `req.body` ;
2. convertir edad a número;
3. comprobar que los campos estén completos y la edad sea válida;
4. ante error, responder 400 y renderizar el formulario;
5. mostrar un mensaje de error con `role="alert"` ;
6. conservar los valores ingresados;
7. ante éxito, generar un identificador;
8. asignar `/img/mascota.svg` como imagen;
9. agregar al arreglo;
10. redirigir a `/mascotas` .

No escribir el nuevo registro en el archivo JSON.

8. Layout y parciales

El layout debe contener:

- `<!doctype html>` ;
- idioma español;
- codificación y viewport;

- título dinámico escapado;
- enlace al CSS;
- encabezado parcial;
- `<main>` con `<%= body %>` ;
- pie parcial;
- referencia al JavaScript estático.

El encabezado debe incluir navegación a inicio, catálogo y formulario. El pie debe aparecer en todas las páginas.

Los datos de mascotas deben mostrarse con `<%= ... %>` . La salida sin escapar debe reservarse para `body` e includes controlados.

9. Recursos estáticos

La aplicación debe servir:

- una hoja de estilos propia;
- una imagen SVG local;
- un archivo JavaScript con al menos un mensaje de consola.

URLs esperadas:

```
/css/estilos.css  
/img/mascota.svg  
/js/app.js
```

No escribir `/public` en las URLs del HTML.

El CSS debe incluir como mínimo:

- tipografía y colores generales;
- contenedor de ancho limitado;
- navegación;
- grilla adaptable;
- tarjetas;
- formulario;
- botón o enlace principal;
- mensaje de error;
- estilo visible de foco.

No se evalúa el uso de frameworks CSS ni una apariencia profesional.

10. Matriz de pruebas

Caso	Estado esperado	Evidencia
Inicio	200	Título y enlace al catálogo
Listado	200	Cinco mascotas iniciales
Estado vacío	200	Mensaje alternativo
Detalle válido	200	Datos completos e imagen
Detalle inexistente	404	Página HTML de error
Formulario	200	Controles etiquetados
Envío incompleto	400	Mensaje y valores conservados
Edad inválida	400	Mensaje y valores conservados
Envío válido	302 seguido de 200	Nueva tarjeta
CSS	200	Estilos aplicados
SVG	200	Imagen visible
JavaScript	200	Mensaje en consola
Reinicio	200	Regreso a cinco registros

Para comprobar el estado vacío se puede entregar temporalmente un arreglo vacío a la vista. Restaurar el código antes de entregar.

11. README

Incluir:

```
# Trabajo práctico 04

## Descripción

## Instalación

## Ejecución

## Páginas y rutas

## Estructura de vistas

## Recursos estáticos

## Formulario

## Persistencia de los datos
```

Explicar:

- diferencia entre layout, vista y parcial;
- datos enviados a una vista mediante `res.render` ;
- función de `express.static` ;
- función de `express.urlencoded` ;
- recorrido POST, redirección y GET;

- motivo por el cual el nuevo registro desaparece al reiniciar.

12. Criterios de evaluación

Criterio	Puntaje
Configuración de Express, EJS y carga inicial	1 punto
Layout, parciales y navegación	1,5 puntos
Página inicial y listado con estado vacío	1,5 puntos
Detalle y página HTML 404	1,5 puntos
Formulario, validación y valores conservados	2 puntos
Creación en memoria y redirección	1 punto
CSS, SVG y JavaScript estáticos	0,75 puntos
README, accesibilidad básica y repositorio	0,75 puntos
Total	10 puntos

13. Fuera de alcance

No se evaluarán:

- routers o controladores;
- middleware personalizado;
- JavaScript interactivo;
- subida de imágenes;
- edición o eliminación;
- persistencia;
- base de datos;
- autenticación;
- sanitización avanzada;
- CSRF.

Agregar funcionalidades fuera de alcance no reemplaza una página o flujo obligatorio incompleto.

14. Comprobación final

Antes de enviar:

- ejecutar `npm run check` ;
- iniciar con `npm start` ;
- recorrer todas las páginas mediante enlaces;
- comprobar estados 400 y 404;
- confirmar la redirección posterior al POST;
- verificar CSS, SVG y JavaScript;
- reiniciar y comprobar la memoria temporal;

- revisar que las variables se muestren escapadas;
- verificar `package-lock.json` y ausencia de `node_modules` ;
- restaurar todos los cambios temporales de prueba.